

By graduation you should have:

- ✓ Python
- ✓ SQL
- ✓ 500+ LeetCode problems
- ✓ GitHub portfolio
- ✓ 4–5 AIML projects
- ✓ Deep Learning
- ✓ LLMs & AI Agents
- ✓ AWS basics
- ✓ Internship experience
- ✓ Strong communication

```
print("Total Reward:", total_reward)

env.close()
```

Training Completed!

Q-Table:

```
[[0.74 0.77 0.7 0.74]
 [0.74 0. 0.46 0.56]
 [0.67 0. 0. 0. ]
 [0. 0. 0. 0. ]
 [0.77 0.81 0. 0.74]
 [0. 0. 0. 0. ]
 [0. 0.9 0. 0. ]
 [0. 0. 0. 0. ]
 [0.81 0. 0.86 0.77]
 [0.81 0.9 0.9 0. ]
 [0.86 0.95 0. 0.83]
 [0. 0. 0. 0. ]
 [0. 0. 0. 0. ]
 [0. 0.9 0.95 0.86]
 [0.9 0.95 1. 0.9 ]
 [0. 0. 0. 0. ]]
```

Optimal Path:

```
State: 0 Action: 1 Next State: 4
State: 4 Action: 1 Next State: 8
State: 8 Action: 2 Next State: 9
State: 9 Action: 1 Next State: 13
State: 13 Action: 2 Next State: 14
State: 14 Action: 2 Next State: 15
```

Goal Reached!
Total Reward: 1

[]:

Run Save Up Down Copy Paste

```

count = [0, 0, 0, 0]
total_reward = 0

for i in range(trials):

    if random.random() < epsilon:
        action = random.randint(0, 3)
    else:
        action = np.argmax(q)

    reward = 1 if random.random() < true_rewards[action] else 0

    count[action] += 1

    q[action] = q[action] + (reward - q[action]) / count[action]

    total_reward += reward

print("\nEpsilon =", epsilon)
print("Estimated Rewards:", np.round(q, 2))
print("Machine Selected:", count)
print("Total Reward:", total_reward)

```

Epsilon = 0.1
Estimated Rewards: [0.17 0.5 0.8 0.5]
Machine Selected: [18, 119, 351, 12]
Total Reward: 351

Epsilon = 0.3
Estimated Rewards: [0.17 0.5 0.77 0.43]
Machine Selected: [41, 32, 385, 42]
Total Reward: 336

Epsilon = 0.5
Estimated Rewards: [0.29 0.5 0.82 0.33]
Machine Selected: [59, 86, 304, 51]
Total Reward: 326

[]:



FileEditViewRunKernelSettingsHelpTrust

+

✂

📄

📄

▶

■

🔄

⏩

Code

▼

JupyterLabPython [conda env:base] *Anaconda Toolbox

```
if energy == 0:
    break

if move == 'Right' and col < 2:
    col += 1
elif move == 'Left' and col > 0:
    col -= 1
elif move == 'Up' and row > 0:
    row -= 1
elif move == 'Down' and row < 2:
    row += 1

energy -= 1

if grid[row][col] == 'X':
    reward -= 20
    print("Obstacle Hit")
elif (row, col) == goal:
    reward += 100
    print("Goal Reached")
    break
else:
    reward -= 1

print("Position:", (row, col), "Energy:", energy, "Reward:", reward)

print("\nFinal Reward:", reward)
print("Remaining Energy:", energy)

Position: (0, 1) Energy: 9 Reward: -1
Position: (0, 2) Energy: 8 Reward: -2
Position: (1, 2) Energy: 7 Reward: -3
Goal Reached

Final Reward: 97
Remaining Energy: 6
```

[]:

🌱

🔍

🏠

⬆

⬇

⬆

🗑

FileEditViewRunKernelSettingsHelp

JupyterLabPython [conda env:base] *Anaconda Toolbox

```
for move in moves:

    if energy == 0:
        break

    if move == 'Right' and col < 2:
        col += 1
    elif move == 'Left' and col > 0:
        col -= 1
    elif move == 'Up' and row > 0:
        row -= 1
    elif move == 'Down' and row < 2:
        row += 1

    energy -= 1

    if grid[row][col] == 'X':
        reward -= 20
        print("Obstacle Hit")
    elif (row, col) == goal:
        reward += 100
        print("Goal Reached")
        break
    else:
        reward -= 1

    print("Position:", (row, col), "Energy:", energy, "Reward:", reward)

print("\nFinal Reward:", reward)
print("Remaining Energy:", energy)
```

Position: (0, 1) Energy: 9 Reward: -1
Position: (0, 2) Energy: 8 Reward: -2
Position: (1, 2) Energy: 7 Reward: -3
Goal Reached

Final Reward: 97
Remaining Energy: 6

[]:

File Edit View Run Kernel Settings Help

Truste

Python [conda env:base] * Code

JupyterLab Python [conda env:base] * Anaconda Toolbox

```
else:
    break

print("Battery Limit Reached")
print("Total Deliveries:", deliveries)
print("Final Reward:", reward)
```

```
Action: Deliver
Battery Left: 25
Deliveries: 1
Reward: 10
-----
Action: Deliver
Battery Left: 20
Deliveries: 2
Reward: 20
-----
Action: Deliver
Battery Left: 15
Deliveries: 3
Reward: 30
-----
Action: Deliver
Battery Left: 10
Deliveries: 4
Reward: 40
-----
Action: Deliver
Battery Left: 5
Deliveries: 5
Reward: 50
-----
Action: Deliver
Battery Left: 0
Deliveries: 6
Reward: 60
-----
Battery Limit Reached
Total Deliveries: 6
Final Reward: 60
```

[]:

Run Stop Up Down Copy Paste

Trusted

JupyterLab Python [conda env:base] * Anaconda Toolbox

[]:

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python [conda env:base] Anaconda Toolbox

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()

model.add(Dense(16, input_shape=(3,), activation='relu'))
model.add(Dense(16, activation='relu'))
model.add(Dense(3, activation='linear'))

model.compile(optimizer='adam', loss='mse')

states = np.array([
    [20,15,10],
    [30,5,10],
    [10,25,20],
    [15,10,30]
])

targets = np.array([
    [10,2,1],
    [10,1,2],
    [1,10,2],
    [2,1,10]
])

model.fit(states, targets, epochs=50, verbose=0)

test = np.array([[18,12,25]])

prediction = model.predict(test, verbose=0)

print("Bandwidth:", test[0])
print("Q Values:", prediction[0])
print("Selected Path:", np.argmax(prediction)+1)

Bandwidth: [18 12 25]
Q Values: [1.0706129 0.6231891 1.3373541]
Selected Path: 3
```

[]:

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python [conda env:base] * Anaconda Toolbox

```
if random.random() < epsilon:
    action = random.randint(0, 3)
else:
    action = q.index(max(q))

reward = 1 if random.random() < true_rewards[action] else 0

count[action] += 1

q[action] = q[action] + (reward - q[action]) / count[action]

greedy_reward += reward

print("Cumulative Reward:", greedy_reward)

print("\nComparison")

if greedy_reward > random_reward:
    print("Epsilon-Greedy performs better.")
else:
    print("Random Action performs better.")

Random Action Selection
Cumulative Reward: 235

Epsilon-Greedy Action Selection
Cumulative Reward: 351

Comparison
Epsilon-Greedy performs better.
```

[]: 

```
battery -= 5

if grid[row][col] == "X":
    reward -= 20
    print("Entered No-Fly Zone")
    break

elif grid[row][col] == "G":
    reward += 20
    print("Delivery Successful")
    break

else:
    reward -= 1

print("Position:", row, col)
print("Battery:", battery)
print("Reward:", reward)
print("-----")

print("Final Reward:", reward)
print("Battery Left:", battery)
```

```
Position: 0 1
Battery: 25
Reward: -1
-----
Position: 0 2
Battery: 20
Reward: -2
-----
Position: 1 2
Battery: 15
Reward: -3
-----
Delivery Successful
Final Reward: 17
Battery Left: 10
```

```
Step: 3
Action: Medium
Consumption: 60
Reward: 30
-----
Step: 4
Action: Low
Consumption: 70
Reward: 40
-----
Step: 5
Action: Medium
Consumption: 90
Reward: 50
-----
Step: 6
Action: Low
Consumption: 100
Reward: 60
-----
Step: 7
Action: Low
Consumption: 100
Reward: 45
-----
Step: 8
Action: High
Consumption: 100
Reward: 30
-----
Step: 9
Action: Low
Consumption: 100
Reward: 15
-----
Step: 10
Action: Medium
Consumption: 100
Reward: 0
-----
Final Consumption: 100
Total Reward: 0
```